
STRUCTURE AND ORGANISATION MANUAL

Unitree L1 Structure and Organisation

Minimal Docker, ROS 2, and OctoMap project
Files, processes, ownership, and data

Simplified OctoMap edition
Prepared 3 August 2026
Validated project baseline simplified OctoMap working-tree baseline
Audience: Operators, maintainers, and reviewers

DOCUMENT CONTROL

This document is generated from `structure-and-organisation.md`.
Commands and paths describe the repository state identified above.

Operate the powered LiDAR only after reading the safety section.

This is the source of truth for file placement, ownership, process boundaries, and generated data in the simplified Unitree L1 project. Technical and safety detail belongs in the engineering manual; step-by-step operation belongs in the user manual.

The project exposes normal Docker and ROS 2 commands. Daily use does not pass through project-specific launch or recording wrappers.

1. Design Contract

1.1 One container, two project ROS packages

The repository has one Compose service, `ros`, and two small project ROS packages: `l1_bringup` and `l1_octomap_bringup`. The image contains ROS 2 Humble, the pinned Unitree driver, the released OctoMap server, RViz2, `rosbag2`, and both `bringup` packages. The service starts with:

```
sleep infinity
```

The idle command keeps the container available for explicit `docker compose exec` sessions. It starts no ROS node. Inside a login shell, the normal stationary mapping command is:

```
ros2 launch l1_octomap_bringup unitree_l1_octomap.launch.py \
  port:=/dev/unitree_lidar \
  monitor:=true \
  rviz:=true
```

The raw alternative is `ros2 launch l1_bringup unitree_l1.launch.py`. The two launches are alternatives because each starts the serial driver.

Recording remains a separate, visible command in a second login shell:

```
ros2 bag record -o /workspace/bags/l1_<name> \
  /unilidar/cloud \
  /unilidar/imu
```

1.2 Runtime boundary

The host supplies Docker, X11, DRI, the serial device, and persistent bag storage. ROS 2 and every project ROS process run in the container.

```
host: Docker + X11 + DRI + serial device + bags/
|
`-> container service ros
    +-- /dev/unitree_lidar
    +-- ROS 2 Humble and installed packages
    +-- explicit driver, monitor, OctoMap, RViz2, and recorder processes
```

RViz2 is a container process although its window appears on the host. Bags are container output made persistent through the host bind mount.

1.3 Provenance

Project source and the three reviewed PDFs are versioned. Docker objects, ROS logs, caches, and bags are generated. X11, Xauthority, DRI, and the serial adapter are host-owned resources. The Unitree SDK is a pinned upstream input, not a project-owned checkout.

The SDK pin is release `v1.0.16`, commit `1bd7d95d8ab7ce7a22058d2bb07e39fd62612aa6`. The Docker build retrieves that exact source and builds its ROS 2 driver into the image. OctoMap comes from the released ROS 2 Humble `octomap_server` apt package; the project does not keep an upstream OctoMap source checkout.

2. Complete File Inventory

2.1 Canonical tree

The final source and publication tree contains 27 project-owned files:

```
.
|-- .dockerignore
|-- .gitignore
|-- LICENSE
|-- README.md
|-- compose.yaml
|-- docker/
|   |-- Dockerfile
|   '-- ros2-profile.sh
-- ros2_ws/
|   '-- src/
|       |-- ll_bringup/
|           |-- CMakeLists.txt
|           |-- package.xml
|           |-- launch/
|               '-- unitree_ll.launch.py
|           '-- config/
|               |-- unitree_ll.yaml
|               '-- unitree_ll.rviz
|       '-- ll_octomap_bringup/
|           |-- CMakeLists.txt
|           |-- package.xml
|           |-- launch/
|               '-- unitree_ll_octomap.launch.py
|           '-- config/
|               |-- octomap.yaml
|               '-- ll_octomap.rviz
-- bags/
|   '-- .gitkeep
-- docs/
|   '-- manuals/
|       |-- README.md
|       |-- engineering-manual.md
|       |-- user-manual.md
|       '-- structure-and-organisation.md
-- scripts/
|   |-- build-manuals.sh
|   '-- render-manual.py
-- exports/
|   '-- manuals/
|       |-- UNITREE_L1_ENGINEERING_MANUAL.pdf
|       |-- UNITREE_L1_USER_MANUAL.pdf
|       '-- UNITREE_L1_STRUCTURE_AND_ORGANISATION.pdf
```

Git does not store directories by themselves. `bags/.gitkeep` preserves the empty bag destination in a new clone; the directory would otherwise appear only after the first recording.

2.2 Root and container files

Path	Responsibility
<code>.dockerignore</code>	Limits the Docker build context to files required by the image and excludes bags, publications, Git metadata, and local output
<code>.gitignore</code>	Excludes ROS build products, bag recordings, caches, and other machine-local state while retaining the three reviewed PDFs
<code>LICENSE</code>	States the license for project-owned material
<code>README.md</code>	Provides the shortest build, start, launch, record, and shutdown path
<code>compose.yaml</code>	Defines the single <code>ros</code> service, runtime devices, X11 access, non-root user, idle command, and bag bind mount
<code>docker/Dockerfile</code>	Builds the Humble image, retrieves the pinned Unitree SDK, installs RViz2 and released OctoMap, and builds the project ROS overlay
<code>docker/ros2-profile.sh</code>	Sources ROS 2 Humble and the image-built overlay for <code>bash -l</code> ; it sets shell state and is not a background process

`compose.yaml` is the complete runtime definition. There are no Compose overlays to combine and no project shell launcher that chooses files on the operator's behalf.

2.3 ROS package files

Path	Responsibility
ros2_ws/src/l1_bringup/CMakeLists.txt	Installs the launch file and configuration files as an <code>ament_cmake</code> package
ros2_ws/src/l1_bringup/package.xml	Declares the package identity, license, build tool, and runtime ROS dependencies
ros2_ws/src/l1_bringup/launch/unitree_l1.launch.py	Composes the Unitree driver, optional cloud-rate monitor process, and optional RViz2 process
ros2_ws/src/l1_bringup/config/unitree_l1.yaml	Holds the driver parameters, topic names, frame names, scan count, scaling, bias, and range limits
ros2_ws/src/l1_bringup/config/unitree_l1.rviz	Defines the fixed frame, cloud display, topic, QoS, grid, axes, point style, and camera
ros2_ws/src/l1_octomap_bringup/CMakeLists.txt	Installs the combined mapping launch and its configuration as an <code>ament_cmake</code> package
ros2_ws/src/l1_octomap_bringup/package.xml	Declares the mapping package identity and runtime dependency on <code>l1_bringup</code> , <code>OctoMap</code> , <code>launch</code> , and <code>RViz2</code>
ros2_ws/src/l1_octomap_bringup/launch/unitree_l1_octomap.launch.py	Includes the raw driver launch, starts <code>octomap_server_node</code> , remaps the cloud input, and optionally starts one map RViz2 process
ros2_ws/src/l1_octomap_bringup/config/octomap.yaml	Holds the stationary map frame, resolution, maximum range, ground-filter choice, and latch setting
ros2_ws/src/l1_octomap_bringup/config/l1_octomap.rviz	Displays the live cloud and occupied <code>MarkerArray</code> in <code>unilidar_lidar</code>

The raw launch owns sensor process composition. The combined launch includes it and adds only OctoMap and its RViz profile. Each YAML and RViz file owns one visible concern. There is no project mapping executable or Python package.

2.4 Data, manuals, and publication tools

Path	Responsibility
bags/.gitkeep	Keeps the otherwise empty persistent bag directory in Git
docs/manuals/README.md	Lists the three manuals and the publication command
docs/manuals/engineering-manual.md	Maintained source for architecture, interfaces, safety, and engineering limits
docs/manuals/user-manual.md	Maintained source for build, operation, visualization, recording, inspection, and shutdown
docs/manuals/structure-and-organisation.md	Maintained source for this file and process inventory
scripts/build-manuals.sh	Manually renders and validates the complete three-PDF publication set
scripts/render-manual.py	Converts the constrained Markdown subset into the shared A4 document format
exports/manuals/UNITREE_L1_ENGINEERING_MANUAL.pdf	Published engineering reference
exports/manuals/UNITREE_L1_USER_MANUAL.pdf	Published operating reference
exports/manuals/UNITREE_L1_STRUCTURE_AND_ORGANISATION.pdf	Published structure reference

The two publication scripts are documentation tools. Docker Compose, the container entry path, the ROS launch file, and rosbag2 never call them. They run only when a maintainer explicitly rebuilds the PDFs.

3. What Runs and When

3.1 Lifecycle phases

Phase	Operator command	What exists
Build	<code>docker compose build</code>	Temporary Docker build steps install packages, clone the exact SDK commit, and run <code>colcon</code> ; the final image persists
Idle	<code>docker compose up -d</code>	One <code>ros</code> service runs Docker init plus <code>sleep infinity</code> ; no ROS node or serial reader exists
Shell	<code>docker compose exec ros bash -l</code>	One interactive login shell sources the ROS profile
Live raw	<code>ros2 launch l1_bringup ...</code>	Driver plus the requested monitor and raw RViz2 children
Live map	<code>ros2 launch l1_octomap_bringup ...</code>	Driver, requested monitor, OctoMap server, and map RViz2 child
Record	<code>ros2 bag record ... in a second shell</code>	A recorder subscribes and writes through the <code>/workspace/bags</code> bind
Stop	<code>Ctrl+C, exit, then docker compose down</code>	ROS children and container stop; image and host bags remain

The build does not open the LiDAR. At idle Compose has mapped X11, DRI, the serial device, and `bags/`, but Docker init and `sleep infinity` open none of them. Docker init is PID 1; it forwards stop signals and reaps children. `docker/ros2-profile.sh` is sourced during `bash -l`; it is not a persistent process. At container startup, Docker init briefly invokes the official `/ros_entrypoint.sh`; that script sources the ROS base environment and then replaces itself with `sleep infinity`. Sourcing a ROS setup file can briefly run `_local_setup_util.py`. These environment-setup processes disappear before the steady idle state.

3.2 Exact live process set

The complete live process set includes:

Process	Condition	Purpose
<code>/sbin/docker-init</code>	Container is up	PID 1; supervises the idle container
<code>sleep infinity</code>	Container is up	Keeps the service available for explicit shells
<code>bash -l</code>	Interactive session	Owens the operator terminal and ROS environment
<code>ros2 launch</code>	Always during live run	Starts, observes, and stops launch children
<code>unitree_lidar_ros2_node</code>	Always	Opens the serial port and publishes cloud and IMU messages
<code>ros2 topic hz /unilidar/cloud</code>	<code>monitor:=true</code>	Prints the point-cloud rate without changing sensor messages
<code>octomap_server_node</code>	Combined map launch only	Subscribes to the cloud and updates the occupancy tree
<code>rviz2</code>	<code>rviz:=true</code>	Renders the raw or combined cloud/map profile through the host display

Exactly one of the raw and map launches should run at a time. Both include the Unitree driver and would otherwise compete for `/dev/unitree_lidar`.

The primary interfaces are:

Name	Type	Producer or consumer
<code>/unilidar/cloud</code>	<code>sensor_msgs/msg/PointCloud2</code>	Driver produces; monitor, OctoMap, RViz2, and recorder consume
<code>/unilidar/imu</code>	<code>sensor_msgs/msg/Imu</code>	Driver produces; recorder consumes
<code>/occupied_cells_vis_array</code>	<code>visualization_msgs/msg/MarkerArray</code>	OctoMap produces; map RViz2 consumes
<code>/octomap_binary</code>	<code>octomap_msgs/msg/OctoMap</code>	OctoMap produces the compact occupancy tree
<code>unilidar_lidar</code>	ROS frame	Cloud frame and fixed stationary map frame, so no TF publisher runs
<code>unilidar_imu</code>	ROS frame	Frame named in IMU message headers

Recording adds a second `bash -l`, the `ros2 bag record` process, and its recorder node. It is not a launch child. Stop the recorder first so metadata is finalized, then stop the launch, exit shells, and run `docker compose down`.

An ordinary ROS graph-inspection command can also start the standard ROS 2 CLI daemon. It is tooling, not a project application. Use the documented `--no-daemon` inspection forms when an exact minimal process audit is desired.

4. Data and Generated State

4.1 Sensor data path

The complete live path is short:

```
host serial device
-> /dev/unitree_lidar in container
-> unitree_lidar_ros2_node
    -> /unilidar/cloud -> octomap_server_node -> occupied map -> RViz2
    |
    | -> RViz2
    | -> monitor
    | -> rosbag2
    | -> rosbag2
    -> /unilidar/imu -> rosbag2
```

The launch terminal is the operational log for the driver, monitor, OctoMap, and RViz2. The recording terminal is the operational log for rosbag2. No project daemon collects or hides these messages.

4.2 Tracked and ignored content

Path or object	Owner	Git policy
Project source and Markdown manuals	Project	Tracked
Three named PDFs	Project publication	Tracked
bags/.gitkeep	Project	Tracked
bags/<recording>/	Operator	Ignored
ROS build/, install/, and log/ if created locally	Build tools	Ignored
Python caches and editor files	Local tools	Ignored
Docker image, container, network, and build cache	Docker Engine	Outside Git
X11 socket, Xauthority, DRI, and serial device	Host OS	Outside Git

A bag directory normally contains `metadata.yaml` and one or more storage files. These are measurement data, not source code. Copy or archive them according to the experiment's data policy, but do not commit them to this repository.

The three PDFs are the only generated publication exception. Their Markdown sources remain authoritative. A PDF change should correspond to an intentional source or renderer change and a successful full manual build.

4.3 Bind and device boundaries

The bag mount is read-write because the recorder must persist data. X11 authorization and the X11 socket are host display resources and should be mounted read-only. DRI and the serial adapter are device mappings, not ordinary files copied into the image.

SAFETY / WARNING

Do not replace the narrow device mappings with a privileged container or broad host-device access. Do not weaken host serial permissions globally merely to make the container start.

The container runs as non-root UID 1000. Files created through the bag bind therefore remain usable by the normal host account that owns the project.

4.4 Publications are independent

Manual publication is an independent maintenance activity:

```
./scripts/build-manuals.sh
```

That command runs the Markdown renderer and PDF validation tools. It does not build the Docker image, start the `ros` service, access the LiDAR, start ROS 2, open RViz2, or record a bag. Conversely, none of the live commands rebuilds a manual.

5. Maintenance Rules

5.1 Put each change in one place

Change	Authoritative file
Container packages, ROS base, Unitree URL or commit	<code>docker/Dockerfile</code>
Runtime device, display, user, command, or bag mount	<code>compose.yaml</code>
Login-shell ROS environment	<code>docker/ros2-profile.sh</code>
Driver parameter, topic, frame, scale, bias, range, or scan count	<code>ros2_ws/src/l1_bringup/config/unitree_l1.yaml</code>
Which ROS processes launch and which arguments enable them	<code>ros2_ws/src/l1_bringup/launch/unitree_l1.launch.py</code>
RViz fixed frame, display, topic, QoS, point style, or camera	<code>ros2_ws/src/l1_bringup/config/unitree_l1.rviz</code>
Package dependency or metadata	<code>ros2_ws/src/l1_bringup/package.xml</code>
Package installation rule	<code>ros2_ws/src/l1_bringup/CMakeLists.txt</code>
OctoMap process composition or launch arguments	<code>ros2_ws/src/l1_octomap_bringup/launch/unitree_l1_octomap.launch.py</code>
OctoMap frame, resolution, range, or filter settings	<code>ros2_ws/src/l1_octomap_bringup/config/octomap.yaml</code>
Map RViz displays, topics, QoS, fixed frame, or camera	<code>ros2_ws/src/l1_octomap_bringup/config/l1_octomap.rviz</code>
OctoMap package dependency, metadata, or installation	<code>ros2_ws/src/l1_octomap_bringup/package.xml</code> and <code>CMakeLists.txt</code>
Operating procedure	<code>docs/manuals/user-manual.md</code>
Engineering contract or limit	<code>docs/manuals/engineering-manual.md</code>
File ownership or process boundary	<code>docs/manuals/structure-and-organisation.md</code>

Do not patch the pinned vendor checkout inside a temporary Docker build. A required upstream change must be represented by an intentional version update or a documented project-owned adaptation.

5.2 Review after a change

Use direct inspection before operation:

```
git status --short
docker compose config
docker compose build
docker compose up -d
docker compose ps
docker compose top ros
```

Inside a login shell, the following show the actual live graph and process set:

```
ros2 node list --no-daemon
ros2 topic list --no-daemon
ps -eo pid,ppid,comm,args --forest
```

These checks support the project's transparency goal: the declared Compose service, operating-system processes, ROS nodes, topics, and stored data should agree.

5.3 Reconstruct from source

A clean reconstruction requires only the project source, Docker, network access for the image build, the required host devices, and an X11 session for RViz2:

```
git clone <project-repository>
cd unitree_l1_project
docker compose build
docker compose up -d
docker compose exec ros bash -l
```

The Dockerfile supplies the exact Unitree commit. `CMakeLists.txt` installs the project launch and configuration. The login profile exposes the built packages. No copied `build/` or `install/` tree is required in Git.

5.4 Final organisation principles

- Keep one Compose service because the live use case needs one container.
- Keep ROS behavior in the ROS package and invoke it with ROS commands.
- Keep sensor and OctoMap parameters separate from launch composition and

RViz display.

- Keep stationary OctoMap honest: the sensor frame is the map frame and no localization or fake TF is hidden in the project.

- Keep recording explicit and independent from visualization.
- Keep raw bags outside Git and reviewed manuals inside Git.
- Keep upstream code pinned and project adaptations project-owned.
- Document every persistent file and every expected runtime process.